# EXPERIMENT NO: 07

## Online Beauty Parlour Appointment

### TITLE

Design and develop an Online Beauty Parlour Appointment Booking System using HTML, CSS and JavaScript.

### AIM

To design and implement a responsive online beauty parlour appointment booking website that allows customers to view available services, select a stylist, choose an appointment date and time, enter their details, and submit a booking request with a full-payment or partial-deposit option.

### SOFTWARE REQUIREMENTS

- Operating System: Windows / Linux / macOS
- Front-end: HTML5, CSS3 and JavaScript
- Editor: Visual Studio Code / Notepad++
- Browser: Google Chrome / Microsoft Edge / Firefox
- Hardware: Computer with minimum 4 GB RAM and internet access

### PROBLEM DESCRIPTION

Beauty parlours commonly receive appointments through phone calls or manual registers. This can lead to booking conflicts, difficulty checking stylist availability, and delays in managing customer information. The proposed website provides a simple online booking form. Customers can select a beauty service, stylist, date, time, and payment preference. The website validates the entered details and displays a booking request summary. In a complete real-world system, a server and database would manage real-time availability, customer accounts, payment gateway transactions, reminders, and staff calendars.

### ALGORITHM

1. Start the program.
2. Create the web page structure using HTML.
3. Apply CSS styles to design the header, service cards, booking form, and responsive layout.
4. Display the available beauty services with their prices and duration.
5. Create input fields for customer name and mobile number.
6. Provide selection lists for service, stylist, appointment time, and payment type.
7. Allow the customer to select an appointment date.
8. Set the minimum selectable date to the current date.

9. When the customer submits the form, prevent the default page refresh.
10. Validate the required fields and check that the mobile number contains 10 digits.
11. Read the selected service price and payment option.
12. If the customer selects a 25% deposit, calculate the deposit as service price multiplied by 0.25.
13. If full payment is selected, calculate the full service price; if pay at salon is selected, set the amount due now to zero.
14. Display the booking request details and amount due.
15. Stop the program.

## PROGRAM CODE

Save the following program as beauty_booking.html and open it in a web browser.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>Online
Beauty Parlour Appointment Booking</title>
<style>
* { box-sizing: border-box; }
body {
  margin: 0; font-family: Arial, sans-serif;
background: #f9f5f8; color: #382d38;
} header {   background: white; padding: 18px 6%;
display: flex; justify-content: space-between;
align-items: center; box-shadow: 0 2px 8px #ddd;
}
.logo { color: #914b73; font-size: 25px; font-weight: bold; } nav
a { text-decoration: none; color: #554452; margin: 0 10px; }
.hero {
  max-width: 1200px; margin: 28px auto; padding: 45px;
background: #f2deea; border-radius: 18px;
}
.hero h1 { font-size: 36px; margin: 10px 0; }
main { max-width: 1100px; margin: auto; padding: 0 18px; }
.services {
  display: grid; grid-template-columns: repeat(3, 1fr); gap: 18px; }
.card, .booking {
  background: white; padding: 22px; border-radius: 14px;
  box-shadow: 0 2px 10px #e8dfe6;
}
```

```
.card h3 { color: #914b73; }
.booking { margin: 28px 0; }
.grid { display: grid; grid-template-columns: 1fr 1fr; gap: 15px; }
label { display: block; font-weight: bold; margin: 10px 0 6px; }
input, select {   width: 100%; padding: 11px; border: 1px solid
#d7c9d3;   border-radius: 7px; font-size: 15px;
} button {   background: #914b73; color:
white; border: 0;   padding: 13px 22px; border-
radius: 8px;
  margin-top: 18px; cursor: pointer; font-weight: bold;
}
#message {
  display: none; margin-top: 18px; padding: 15px;
  background: #f2e5ed; border-radius: 8px; line-height: 1.7;
}
footer { text-align: center; padding: 20px; color: #766875; }
@media(max-width:700px) {
header { display: block; }   nav
{ margin-top: 12px; }
  .services, .grid { grid-template-columns: 1fr; }
  .hero h1 { font-size: 27px; }
}
</style>
</head>
<body>
<header>
  <div class="logo">GLOW & GO</div>
  <nav>
    <a href="#">Home</a>
    <a href="#services">Services</a>
    <a href="#booking">Book Appointment</a>
  </nav>
</header>

<section class="hero">
  <h1>Book Your Beauty Appointment</h1>
  <p>Select your favourite service, stylist, date and time.</p>
  <a href="#booking"><button>Book Now</button></a>
</section>

<main>
  <section id="services">
    <h2>Available Beauty Services</h2>
    <div class="services">
      <div class="card"><h3>Hair Styling</h3><p>Price: ₹800</p><p>Duration: 45 minutes</p></div>
```

```
    <div class="card"><h3>Facial</h3><p>Price: ₹1,200</p><p>Duration: 60 minutes</p></div>
<div class="card"><h3>Manicure</h3><p>Price: ₹600</p><p>Duration: 45 minutes</p></div>
</div>
 </section>

 <section class="booking" id="booking">
  <h2>Appointment Booking Form</h2>
  <form id="bookingForm">
   <div class="grid">
    <div>
     <label for="name">Customer Name</label>
     <input id="name" type="text" required placeholder="Enter your name">
</div>
    <div>
     <label for="phone">Mobile Number</label>
     <input id="phone" type="tel" pattern="[0-9]{10}" required placeholder="10 digit number">
</div>
    <div>
     <label for="service">Select Service</label>
     <select id="service" required>
      <option value="">Choose service</option>
      <option value="800">Hair Styling - ₹800</option>
      <option value="1200">Facial - ₹1200</option>
      <option value="600">Manicure - ₹600</option>
     </select>
    </div>
    <div>
     <label for="stylist">Select Stylist</label>
     <select id="stylist" required>
      <option value="">Choose stylist</option>
      <option>Priya</option><option>Ananya</option><option>Meera</option>
     </select>
    </div>
    <div>
     <label for="date">Appointment Date</label>
     <input id="date" type="date" required>
    </div>
    <div>
     <label for="time">Appointment Time</label>
     <select id="time" required>
      <option value="">Choose time</option>
      <option>09:00 AM</option><option>10:00 AM</option>
      <option>11:00 AM</option><option>11:30 AM</option>
      <option>02:00 PM</option><option>03:00 PM</option>
      <option>04:00 PM</option>
     </select>
```

```
        </div>
        <div>
          <label for="payment">Payment Type</label>
          <select id="payment">
            <option value="full">Full Payment</option>
            <option value="deposit">25% Deposit</option>
            <option value="salon">Pay at Salon</option>
          </select>
        </div>
      </div>
      <button type="submit">Submit Booking</button>
    </form>
    <div id="message" aria-live="polite"></div>
  </section>
</main>
<footer>Online Beauty Parlour Appointment Booking System</footer>

<script> const dateField =
document.getElementById("date"); dateField.min =
new Date().toISOString().split("T")[0];

document.getElementById("bookingForm").addEventListener("submit", function(event) {
event.preventDefault();

  const name = document.getElementById("name").value.trim();
const phone = document.getElementById("phone").value.trim();
const service = document.getElementById("service");   const
stylist = document.getElementById("stylist").value;   const date
= dateField.value;   const time =
document.getElementById("time").value;   const payment =
document.getElementById("payment").value;

  if (!/^[0-9]{10}$/.test(phone)) {     alert("Please enter
a valid 10-digit mobile number.");     return;
  }

  const price = Number(service.value);   let amount
= price;   if (payment === "deposit") amount =
price * 0.25;   if (payment === "salon") amount =
0;

  const output = document.getElementById("message");
  output.style.display = "block";
output.textContent =
    "Booking request received (demonstration). Customer: " + name +
    " | Stylist: " + stylist + " | Date: " + date + " | Time: " + time +
```

```
    " | Service price: Rs. " + price + " | Amount due now: Rs. " + amount +
    ". Actual availability and payment must be confirmed by a secure server."; });
</script>
</body>
</html>
```

## SAMPLE INPUT

| Input | Sample Value |
|---|---|
| Customer Name | Ananya Kumar |
| Mobile Number | 9876543210 |
| Service | Hair Styling – ₹800 |
| Stylist | Priya |
| Appointment Date | 12-11-2026 |
| Appointment Time | 11:30 AM |
| Payment Type | 25% Deposit |

## EXPECTED OUTPUT

The website displays the selected booking details after the customer submits valid information. For the sample input, the service price is ₹800 and the 25% deposit is ₹200.

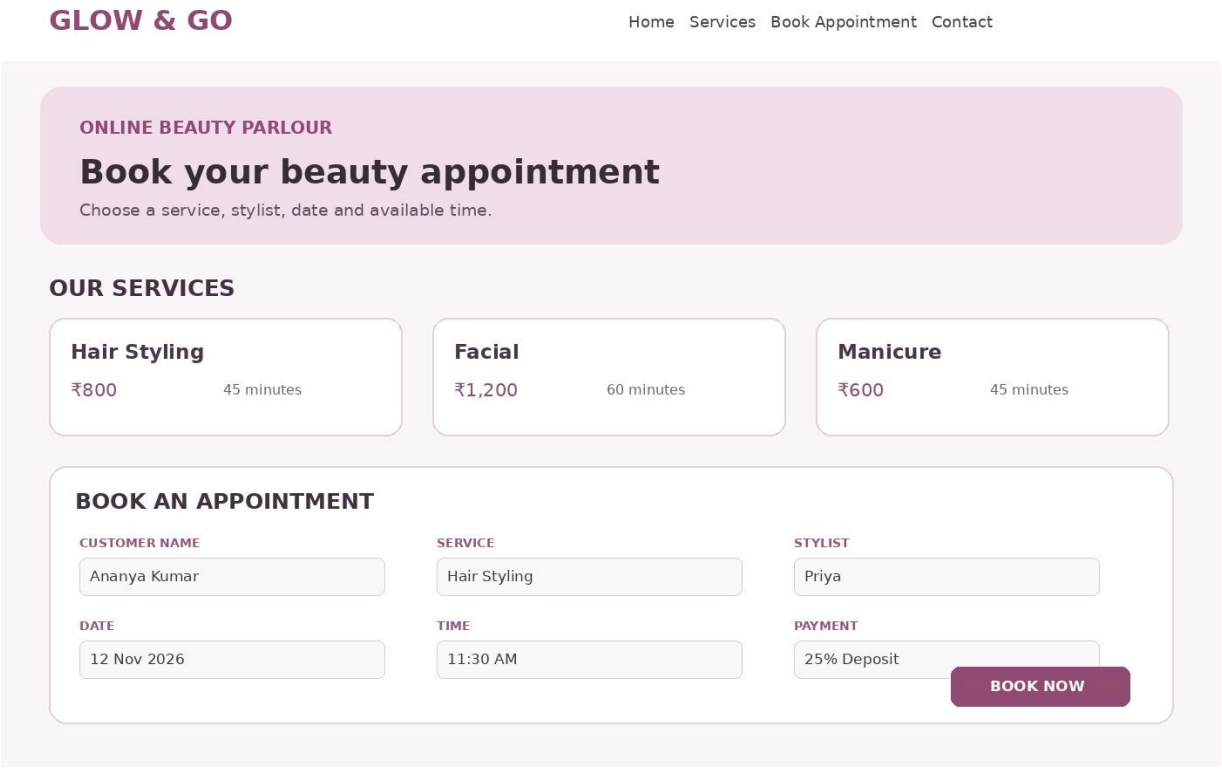



Figure: Sample browser output of the Online Beauty Parlour Appointment Booking System.

## RESULT

Thus, the Online Beauty Parlour Appointment Booking System was designed using HTML, CSS and JavaScript. The web page displays beauty services and provides a booking form to select a stylist, date,

time, and payment option. Form validation and the sample booking output were demonstrated successfully.

# EXPERIMENT NO: 08

## Online Hospital Appointment

### TITLE

Design and develop an Online Hospital Appointment Booking System for MedCare Hospital using HTML, CSS and JavaScript.

### AIM

To design and implement a responsive hospital appointment booking website that allows patients to search hospitals and doctors by location and specialty, view doctor details, select a preferred appointment date and time, and submit an appointment request. The system also provides a foundation for patient registration, appointment management, check-in, reviews, and hospital administration.

### SOFTWARE REQUIREMENTS

Operating System: Windows / Linux / macOS

Front-end Technologies: HTML5, CSS3 and JavaScript

Code Editor: Visual Studio Code / Notepad++

Web Browser: Google Chrome / Microsoft Edge / Firefox

Hardware: Computer with minimum 4 GB RAM

### PROBLEM DESCRIPTION

Patients often need to contact hospitals by phone or visit them in person to find doctors and book appointments. Manual processes can cause long waiting times, paperwork, scheduling conflicts, and difficulty accessing previous appointment information. MedCare Hospital proposes an online platform where patients can register, log in, search hospitals and doctors by location or specialty, view available appointment times, book or cancel appointments, and receive reminders. Hospital staff can manage doctors, schedules, appointments, ratings, and hospital information. The front-end lab prototype demonstrates doctor listing and appointment request submission. Real patient records, authentication, live availability, maps, reminders, and healthcare data protection require secure server-side implementation.

## MODULES

### Search Module

Search for hospitals and doctors by location, specialty, and hospital name.

### Appointment Booking

Select hospital, doctor, date, time, and submit a booking request.

### Check-in Form

Allow a patient to submit check-in details for a confirmed appointment.

### Appointment Management

View, update, or cancel appointments based on hospital policy.

### Schedule Management

Display doctor schedules and make available appointment timings.

### Past Appointment Management

Allow patients to view previous appointments and check-in history.

### Admin Features

Manage appointments, doctors, hospital details, appointment statistics, and ratings.

### Patient Features

Patient signup/login, doctor search, booking, appointment status, reviews, and personal details.

### Hospital Features

Hospital registration/login, create and update hospital details, and update location.

## ALGORITHM

1. Start the program.
2. Create the website structure using HTML.
3. Apply CSS to design the navigation bar, doctor cards, appointment form, and responsive layout.
4. Display the MedCare Hospital home page.
5. Display the available doctor list with specialties, hospital names, and ratings.
6. Allow the patient to enter their name, mobile number, and location.
7. Allow the patient to select a hospital and medical specialty.

8. Allow the patient to select a doctor, appointment date, and time.
9. Set the minimum appointment date to the current date.
10. When the patient submits the form, prevent the default page refresh.
11. Validate all required fields and verify the mobile number contains 10 digits.
12. Read the submitted patient, hospital, doctor, specialty, date, and time details.
13. Display the appointment request summary on the web page.
14. In a complete system, send the request to the server and verify doctor availability before confirmation.
15. Stop the program.

**PROGRAM CODE**

Save the following program as medcare_appointment.html and open it in a web browser.

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>MedCare Hospital Appointment Booking</title>

<style>

* { box-sizing: border-box; } body {

margin: 0; font-family: Arial, sans-serif;

background: #f5f8fc; color: #2d3946;

} header { background: white; padding: 18px 6%;

display: flex; justify-content: space-between; align-

items: center; box-shadow: 0 2px 8px #dce3ea;

}

.logo { font-size: 28px; font-weight: bold; color: #1d4670; } nav

a { margin: 0 10px; text-decoration: none; color: #34495e; }
```

```
.hero { max-width: 1200px; margin: 28px auto; padding: 42px;
border-radius: 18px; background: #e1effa;
}
.hero h1 { color: #1d4670; font-size: 34px; } main { max-
width: 1120px; margin: auto; padding: 0 18px; }
.doctors { display: grid; grid-template-columns: repeat(3, 1fr);
gap: 18px;
}
.card, .booking { background: white; padding: 22px;
border-radius: 14px; box-shadow: 0 2px 10px #e0e7ee;
}
.card h3 { color: #1d4670; } .card p { margin:
8px 0; } button { background: #3074ae;
color: white; border: 0; border-radius: 7px;
padding: 12px 18px; cursor: pointer; font-
weight: bold;
}
.booking { margin: 28px 0; }
.grid { display: grid; grid-template-columns: 1fr 1fr; gap: 14px; }
label { display: block; margin: 10px 0 6px; font-weight: bold; }
input, select, textarea { width: 100%; padding: 11px; border: 1px
solid #cbd7e2; border-radius: 7px; font-size: 15px;
}
#result { display: none; margin-top: 18px; padding: 15px;
background: #e8f2fa; border-radius: 8px; line-height: 1.7;
} footer { text-align: center; padding: 22px; color: #607080;
}
```

```
@media(max-width:700px) { header { display:
block; } nav { margin-top: 12px; }
.doctors, .grid { grid-template-columns: 1fr; }
.hero h1 { font-size: 27px; }
}
</style>
</head>
<body>
<header>
<div class="logo">MedCare Hospital</div>
<nav>
<a href="#">Home</a><a href="#doctors">Find Doctors</a>
<a href="#booking">Book Appointment</a><a href="#contact">Contact</a>
</nav>
</header>
<section class="hero">
<h1>Find the Right Doctor, Book with Ease</h1>
<p>Search hospitals and doctors by specialty and location, then choose a convenient appointment
slot.</p>
<a href="#booking"><button>Book an Appointment</button></a>
</section>
<main>
<section id="doctors">
<h2>Available Doctors</h2>
<div class="doctors">
<div class="card">
<h3>Dr. Priya Sharma</h3><p>Specialty: Cardiology</p>
```

```
<p>Hospital: City Care Hospital</p><p>Rating: 4.8 / 5</p>
</div>
<div class="card">
<h3>Dr. Arun Kumar</h3><p>Specialty: Dermatology</p>
<p>Hospital: MedCare Central</p><p>Rating: 4.6 / 5</p>
</div>
<div class="card">
<h3>Dr. Meena Rao</h3><p>Specialty: Pediatrics</p>
<p>Hospital: Green Valley Hospital</p><p>Rating: 4.9 / 5</p>
</div>
</div>
</section>
<section class="booking" id="booking">
<h2>Patient Appointment Form</h2>
<form id="appointmentForm">
<div class="grid">
<div>
<label for="patient">Patient Name</label>
<input id="patient" type="text" required placeholder="Enter patient name">
</div>
<div>
<label for="phone">Mobile Number</label>
<input id="phone" type="tel" pattern="[0-9]{10}" required placeholder="10 digit mobile number">
</div>
<div>
<label for="location">Location</label>
<input id="location" type="text" required placeholder="Enter city or area">
```

```
</div>
<div>
<label for="hospital">Select Hospital</label>
<select id="hospital" required>
<option value="">Choose hospital</option>
<option>City Care Hospital</option>
<option>MedCare Central</option>
<option>Green Valley Hospital</option>
</select>
</div>
<div>
<label for="specialty">Select Specialty</label>
<select id="specialty" required>
<option value="">Choose specialty</option>
<option>Cardiology</option><option>Dermatology</option>
<option>Pediatrics</option><option>Orthopedics</option>
<option>General Medicine</option>
</select>
</div>
<div>
<label for="doctor">Select Doctor</label>
<select id="doctor" required>
<option value="">Choose doctor</option>
<option>Dr. Priya Sharma</option>
<option>Dr. Arun Kumar</option>
<option>Dr. Meena Rao</option>
</select>
```

```
</div>
<div>
<label for="date">Appointment Date</label>
<input id="date" type="date" required>
</div>
<div>
<label for="time">Appointment Time</label>
<select id="time" required>
<option value="">Choose time</option>
<option>09:00 AM</option><option>10:00 AM</option>
<option>10:30 AM</option><option>11:00 AM</option>
<option>02:00 PM</option><option>03:00 PM</option>
</select>
</div>
</div>
<label for="reason">Reason for Visit (optional)</label>
<textarea id="reason" rows="3" placeholder="Enter a short reason for the appointment"></textarea>
<button type="submit">Submit Appointment</button>
</form>
<div id="result" aria-live="polite"></div>
</section>
<section id="contact"><h2>Contact</h2><p>Contact the selected hospital for assistance with your appointment.</p></section>
</main>
<footer>© 2026 MedCare Hospital – Online Appointment Booking System</footer>
<script> const dateField = document.getElementById("date"); dateField.min = new
Date().toISOString().split("T")[0];
```

```
document.getElementById("appointmentForm").addEventListener("submit", function(event)
{ event.preventDefault(); const patient = document.getElementById("patient").value.trim();
const phone = document.getElementById("phone").value.trim(); const location =
document.getElementById("location").value.trim(); const hospital =
document.getElementById("hospital").value; const specialty =
document.getElementById("specialty").value; const doctor =
document.getElementById("doctor").value; const date = dateField.value; const time =
document.getElementById("time").value; const reason =
document.getElementById("reason").value.trim(); if (!/^[0-9]{10}$/.test(phone)) {
alert("Please enter a valid 10-digit mobile number."); return; } const result =
document.getElementById("result"); result.style.display = "block"; result.textContent =
"Appointment request submitted (demo). Patient: " + patient +
" | Location: " + location + " | Hospital: " + hospital +
" | Specialty: " + specialty + " | Doctor: " + doctor +
" | Date: " + date + " | Time: " + time +
(reason ? " | Reason: " + reason : "") +
". The hospital must verify live availability and confirm the appointment.";
});
</script>
</body>
</html>
```

**SAMPLE INPUT**

| Input Field | Sample Value |
|---|---|
| Patient Name | Ananya Kumar |
| Mobile Number | 9876543210 |
| Location | Chennai |

| | |
|---|---|
| Hospital | City Care Hospital |
| Specialty | Cardiology |
| Doctor | Dr. Priya Sharma |
| Appointment Date | 12-11-2026 |
| Appointment Time | 10:30 AM |
| Reason for Visit | Regular consultation |

**EXPECTED OUTPUT**

After valid details are entered and the Submit Appointment button is clicked, the browser displays an appointment request summary containing the patient name, location, hospital, specialty, doctor, appointment date, and time. This is a demonstration request; it is not a confirmed hospital appointment.

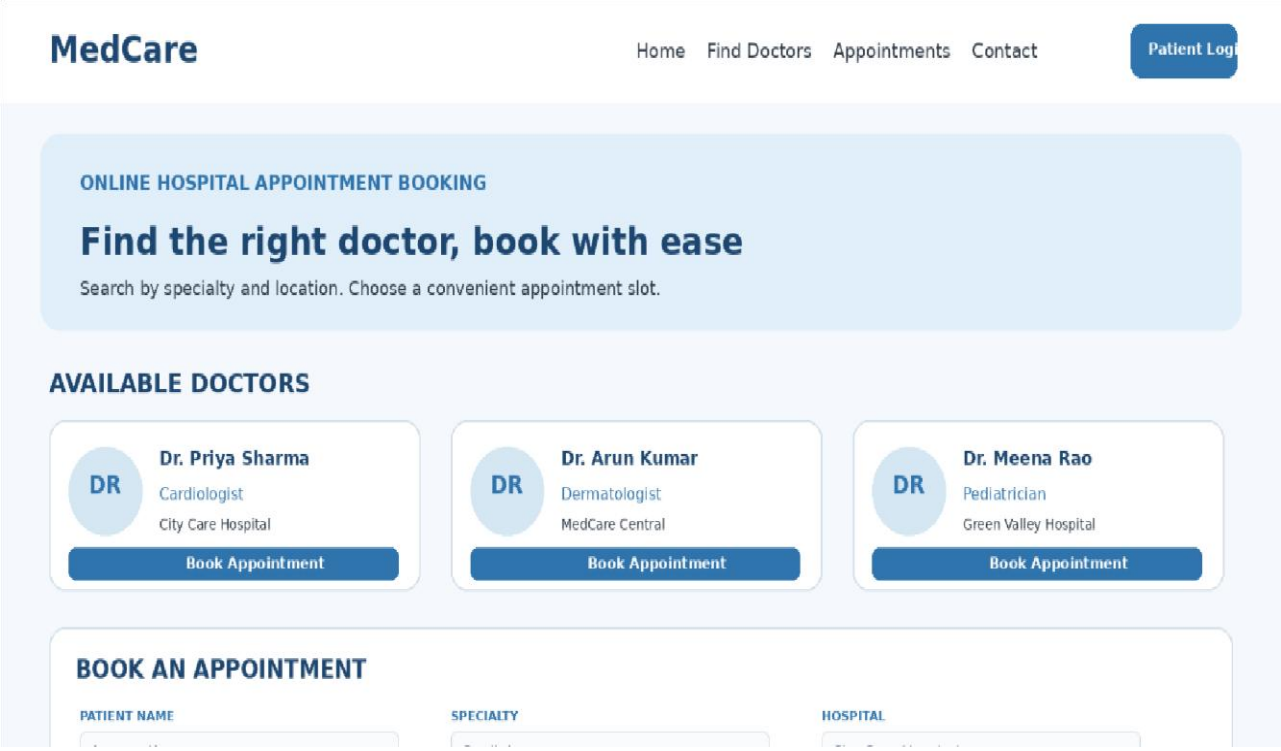



Figure: Sample output screen of MedCare Hospital Appointment Booking System.

**RESULT**

Thus, the MedCare Hospital Online Appointment Booking System was designed using HTML, CSS and JavaScript. The website displays doctor information and provides a patient appointment form with hospital, specialty, doctor, date, and time selection. Input validation and appointment request output were demonstrated successfully.

# Experiment-09

## TechSavvy – Online Shopping Platform

### 1. Title

Design and Development of an Online Shopping Platform for TechSavvy using HTML, CSS and JavaScript.

### 2. Aim

To design and implement a responsive online shopping website that allows customers to browse categorized electronic products, search and filter items, view product details, add or remove products from a shopping cart, and place demo orders. The website also provides a basic administrator panel to add, edit, and delete products.

### 3. Software Requirements

Operating System: Windows 10/11, Linux or macOS

Editor: Visual Studio Code / Notepad++

Languages: HTML5, CSS3 and JavaScript (ES6)

Browser: Google Chrome, Microsoft Edge or Firefox

Storage: Browser Local Storage for this front-end demonstration

Hardware: Computer with at least 4 GB RAM and a modern web browser

### 4. Problem Description

TechSavvy requires an easy-to-use online platform for selling electronic gadgets. Customers should be able to view product categories and their associated products, search for items, check displayed availability, add products to a cart, modify quantities, remove items, and complete a checkout flow. They should also be able to view order status and order history. The administrator must be able to add new products, remove existing products, and update product prices and descriptions. This experiment implements the main shopping workflow as a client-side prototype. Product and order data are stored in the browser's Local Storage so that they remain available after a page refresh on the same browser.

### 5. Modules

Home and Category Module: Displays the TechSavvy home page, popular categories and product subcategories/products.

Product Search and Details Module: Filters products by keyword and category and displays each product's description and price.

Shopping Cart Module: Adds items, changes quantities, removes individual items, clears the cart and calculates the total.

Checkout and Order Module: Collects a demo customer name, creates an order number and records the order with an initial status.

Order Tracking and History Module: Displays previously placed demo orders, dates, totals and current order status.

Administrator Module: Allows the administrator to add, edit and delete product records in the demonstration catalogue.

Login and Logout Demonstration: Provides a simple front-end login prompt; production authentication is not implemented.

**6. Algorithm**

Start the application and load the saved products, cart and orders from Local Storage. If no saved catalogue exists, load the default products.

Display the home page with product categories and the available products.

When the customer enters a search term or selects a category, filter and display matching products.

When Add to Cart is selected, increase the product quantity in the cart or add it as a new cart entry.

Allow the customer to increase or decrease quantities, remove individual items, or clear the cart. Recalculate the total after every change.

When Checkout is selected, verify that the cart is not empty and request the customer's name.

Generate a unique demo order number, save the order details and initial status, then clear the cart.

Display order history and tracking information.

If the administrator adds, edits or deletes a product, validate the input and update the catalogue.

Save all updated data to Local Storage and refresh the displayed sections. Stop.

**7. Complete HTML, CSS and JavaScript Code**

Save the following code as index.html and open it in a modern web browser. The complete code includes the page structure, responsive styling and JavaScript functionality.

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>TechSavvy - Online Shopping</title>

<style>

:root{--navy:#102a43;--blue:#147dbe;--light:#f3f6fb;--muted:#627d98}

*{box-sizing:border-box}body{margin:0;font-family:Arial,sans-serif;background:var(-
light);color:#243b53}

header{background:var(--navy);color:white;padding:16px 5%;display:flex;align-
items:center;justifycontent:space-between;gap:16px;flex-wrap:wrap}

.logo{font-size:28px;font-weight:bold}nav a{color:white;text-decoration:none;margin:0 9px}

input,select,button{font:inherit;padding:10px;border:1px solid #bcccdc;border-radius:7px}

button{background:var(--blue);color:white;border:0;cursor:pointer}button:hover{opacity:.9}

main{max-
width:1200px;margin:auto;padding:24px}.hero{background:#dceeff;padding:38px;borderradius:16px;m
argin-bottom:25px}

.hero h1{font-size:34px;margin:0 0 10px;color:var(--navy)}.section{margin:25px 0}

.grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(220px,1fr));gap:16px}
.card{background:white;padding:18px;border-radius:12px;box-shadow:0 2px 8px #102a4312}

.category{border:1px solid #d9e2ec}.muted{color:var(--muted);font-size:14px}

.price{font-size:19px;font-weight:bold;color:var(--blue)}

.actions{display:flex;gap:8px;flex-wrap:wrap;margin-top:10px}

#cartItems li{margin:9px 0}.admin{background:#fff8e6;padding:18px;border-radius:12px} footer{text-

align:center;padding:25px;color:var(--muted)}

@media(max-width:650px){header{align-items:flex-start}nav{line-height:2}.hero h1{font-size:27px}}

</style>

</head>

<body>

<header>

<div class="logo">TechSavvy</div>
```

```
<nav><a href="#home">Home</a><a href="#categories">Categories</a><a
href="#products">Products</a><a href="#orders">Orders</a><a href="#admin">Admin</a></nav>

<div><input id="search" placeholder="Search gadgets..." oninput="renderProducts()">

<button onclick="showLogin()">Login</button></div>

</header>

<main id="home">

<section class="hero"><h1>Upgrade Your Tech Life</h1>

<p>Discover the latest gadgets at exclusive online prices.</p>

<button onclick="document.getElementById('products').scrollIntoView()">Shop Now</button>

</section>

<section class="section" id="categories"><h2>Shop by Category</h2><div id="categoryList"
class="grid"></div></section>

<section class="section" id="products"><h2>Popular Products</h2>

<label>Category: <select id="categoryFilter" onchange="renderProducts()"><option value="All">All
categories</option></select></label>

<div id="productList" class="grid" style="margin-top:15px"></div>

</section>

<section class="section card"><h2>Shopping Cart</h2><ul id="cartItems"></ul>

<p><b>Total: ₹<span id="total">0</span></b></p>

<div class="actions"><button onclick="checkout()">Checkout</button><button
onclick="clearCart()">Remove All</button></div>

</section>

<section class="section card" id="orders"><h2>Order Tracking & History</h2><div id="orderList">No
orders placed yet.</div></section>

<section class="section admin" id="admin"><h2>Administrator - Manage Products</h2>

<p class="muted">Demo administration panel. In a real deployment, protect this panel with server-side
authentication and authorization.</p>

<div class="actions"><input id="pname" placeholder="Product name"><input id="pprice"
type="number" min="1" placeholder="Price">
```

<select id="pcat"><option>Smartphones</option><option>Laptops</option><option>Tablets</option><option>Accessories</option></select>

<input id="pdesc" placeholder="Description"><button onclick="addProduct()">Add Product</button>

</section>

</main>

<footer>© 2026 TechSavvy Online Shopping Platform</footer>

<script> const

defaultProducts=[

{id:1,name:"Nova X Pro Smartphone",price:49999,category:"Smartphones",description:"5G smartphone with 256 GB storage."},

{id:2,name:"TechBook Air 14",price:74999,category:"Laptops",description:"Lightweight laptop with 16 GB RAM."},

{id:3,name:"TabPlus 11",price:29999,category:"Tablets",description:"11-inch tablet for work and entertainment."},

{id:4,name:"Wireless Buds Z",price:3499,category:"Accessories",description:"Wireless earbuds with noise cancellation."}

];

let products=JSON.parse(localStorage.getItem("ts_products"))||defaultProducts;

let cart=JSON.parse(localStorage.getItem("ts_cart"))||[]; let

orders=JSON.parse(localStorage.getItem("ts_orders"))||[]; const

categories=["Smartphones","Laptops","Tablets","Accessories"];

function save(){localStorage.setItem("ts_products",JSON.stringify(products));localStorage.setItem("ts_cart",JSON.stringify(cart));localStorage.setItem("ts_orders",JSON.stringify(orders))} function init(){

const list=document.getElementById("categoryList"),filter=document.getElementById("categoryFilter"),adminCat=document.getElementById("pcat");

```
list.innerHTML=categories.map(c=>`<div class="card category"><h3>${c}</h3><p
class="muted">${products.filter(p=>p.category===c).map(p=>p.name).join(" • ")||"Explore
products"}</p></div>`).join("");

filter.innerHTML='<option value="All">All

categories</option>'+categories.map(c=>`<option>${c}</option>`).join("");

renderProducts();renderCart();renderOrders();renderAdmin();

} function

renderProducts(){

const
term=document.getElementById("search").value.toLowerCase(),cat=document.getElementById("catego
ryFilter").value;

const
found=products.filter(p=>(cat==="All"||p.category===cat)&&(p.name.toLowerCase().includes(term)||p.
description.toLowerCase().includes(term)));

document.getElementById("productList").innerHTML=found.map(p=>`<article
class="card"><h3>${p.name}</h3><p class="muted">${p.category}</p><p>${p.description}</p><p
class="price">₹${p.price.toLocaleString("en-IN")}</p><div class="actions"><button
onclick="addToCart(${p.id})">Add to Cart</button><button
onclick="alert('${p.name.replace(/'/g,"")}\\n${p.description.replace(/'/g,"")}')">Details</button></div>
</article>`).join("")||"<p>No matching products found.</p>";

}

function addToCart(id){let item=cart.find(x=>x.id===id);if(item)item.qty++;else

cart.push({id,qty:1});save();renderCart();alert("Item added to cart.");} function

renderCart(){

document.getElementById("cartItems").innerHTML=cart.map(x=>{let
p=products.find(y=>y.id===x.id);return p?`<li>${p.name} — ₹${p.price} × ${x.qty} <button
onclick="changeQty(${x.id},-1)">−</button> <button onclick="changeQty(${x.id},1)">+</button>
<button onclick="removeItem(${x.id})">Remove</button></li>`:""}).join("")||"<li>Your cart is
empty.</li>";

document.getElementById("total").textContent=cart.reduce((s,x)=>s+(products.find(p=>p.id===x.id)?.p
rice||0)*x.qty,0).toLocaleString("en-IN");

}

function changeQty(id,n){let

x=cart.find(i=>i.id===id);if(x)x.qty+=n;cart=cart.filter(i=>i.qty>0);save();renderCart()}
```

```
function removeItem(id){cart=cart.filter(x=>x.id!==id);save();renderCart()} function
clearCart(){cart=[];save();renderCart()} function checkout(){
if(!cart.length){alert("Your cart is empty.");return} let customer=prompt("Enter
customer name to place demo order:");if(!customer)return; let
total=cart.reduce((s,x)=>s+(products.find(p=>p.id===x.id)?.price||0)*x.qty,0);
orders.unshift({number:"TS"+Date.now().toString().slice(-7),customer,total,status:"Order
Placed",date:new Date().toLocaleString()}); cart=[];save();renderCart();renderOrders();alert("Demo
order placed successfully.");
}
function
renderOrders(){document.getElementById("orderList").innerHTML=orders.map(o=>`<p><b>${o.num
ber}</b> — ${o.customer} — ₹${o.total.toLocaleString("en-IN")} —
<span>${o.status}</span><br><small>${o.date}</small></p>`).join("")||"No orders placed yet."}
function addProduct(){
let
name=document.getElementById("pname").value.trim(),price=Number(document.getElementById("ppr
ice").value),category=document.getElementById("pcat").value,description=document.getElementById("
pdesc").value.trim(); if(!name||price<=0||!description){alert("Enter product name, a valid price and
description.");return} products.push({id:Date.now(),name,price,category,description});save();init();
document.getElementById("pname").value="";document.getElementById("pprice").value="";document.
getElementById("pdesc").value="";
}
function renderAdmin(){document.getElementById("adminList").innerHTML="<h3>Current
products</h3>"+products.map(p=>`<p>${p.name} — ₹${p.price} <button
onclick="editProduct(${p.id})">Edit</button> <button
onclick="deleteProduct(${p.id})">Delete</button></p>`).join("")}
function editProduct(id){let p=products.find(x=>x.id===id),price=prompt("Enter new
price:",p.price),desc=prompt("Enter new
description:",p.description);if(price!==null&&Number(price)>0&&desc!==null){p.price=Number(price
);p.description=desc;save();init()}}
function deleteProduct(id){if(confirm("Delete this
product?")){products=products.filter(p=>p.id!==id);cart=cart.filter(x=>x.id!==id);save();init()}}
```

function showLogin(){let user=prompt("Enter username (demo login):");if(user)alert("Welcome, "+user+". This is a front-end demonstration login only.")}

init();

</script>

</body>

</html>

**8. Sample Input**

Example actions and values used to test the application:

Search term: Nova X Pro

Category filter: Smartphones

Product selected: Nova X Pro Smartphone

Quantity: 2

Customer name at checkout: Arun

Administrator adds: SmartWatch S1 | Price: 5999 | Category: Accessories | Description: Smart wearable watch.

**9. Expected Output with Image**

The home page displays the TechSavvy navigation bar, search field, promotional banner, product categories and popular products with prices and Add to Cart controls. When products are added, the cart displays item quantities and the calculated total. Checkout creates a demo order that appears in the order history.

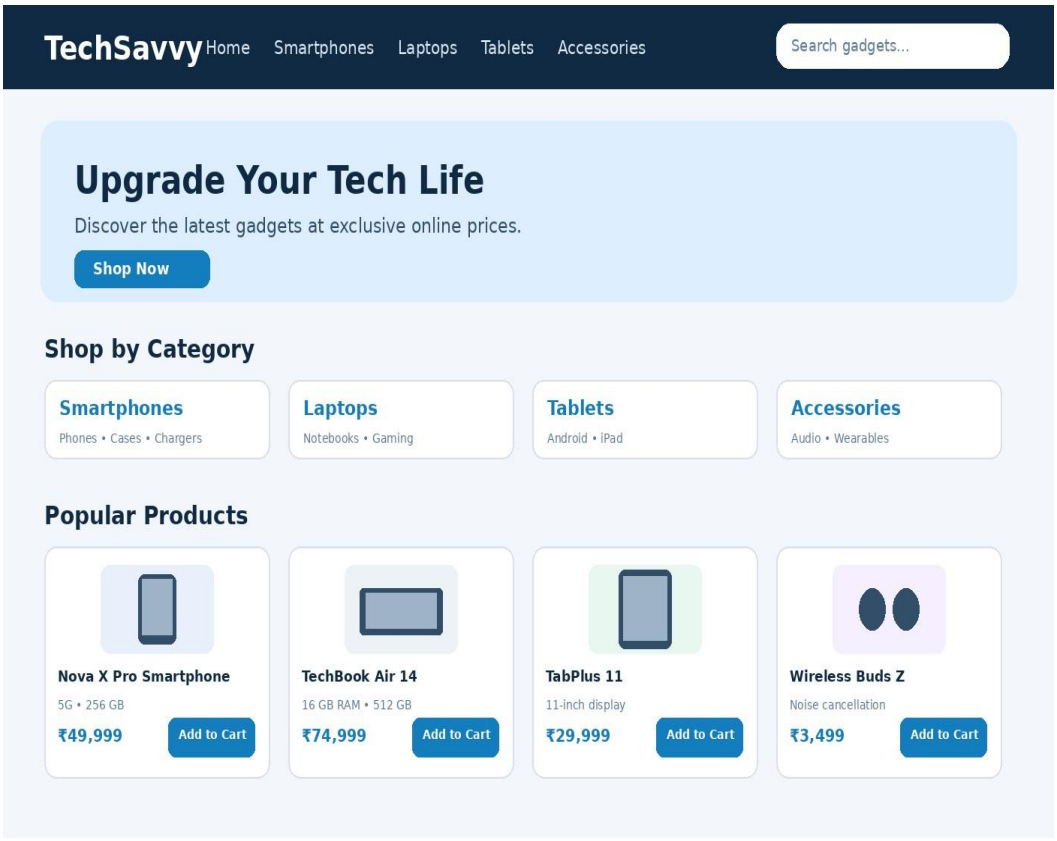



*Figure 1: Expected home page output of the TechSavvy online shopping platform.*

**10. Result**

Thus, the TechSavvy online shopping platform was designed and implemented successfully using HTML, CSS and JavaScript. The application demonstrates product categorization, searching and filtering, product details, shopping cart management, checkout, order history, and administrator product management. Local Storage is used to preserve demonstration data in the browser.

# Experiment-10

# StyleHub  Online Fashion Shopping Platform

## *HTML, CSS and JavaScript*

**1. Title**

Design and Development of an Online Fashion Shopping Platform for StyleHub using HTML, CSS and JavaScript.

**2. Aim**

To develop a responsive online fashion shopping website that displays clothing, footwear and accessories in logical categories. The application enables customers to search and filter products, view descriptions and prices, add or remove products from a cart, and place demonstration orders. It also provides an administrator panel to add, edit and delete catalogue items.

**3. Software Requirements**

Operating System: Windows 10/11, Linux or macOS

Code Editor: Visual Studio Code / Notepad++

Front-end Technologies: HTML5, CSS3 and JavaScript (ES6)

Web Browser: Google Chrome, Microsoft Edge or Firefox

Storage: Browser Local Storage for prototype data

Hardware: Computer with 4 GB RAM or above and internet/browser access

**4. Problem Description**

StyleHub needs a user-friendly e-commerce platform where fashion customers can quickly discover products. The home page should show popular categories first and display related products or subcategories. Customers should be able to search for products, check catalogue availability, inspect item details, add items to a shopping cart, edit quantities, remove items, checkout, and review order status and history. Administrators should be able to add new products, remove products, change prices, update descriptions and review customer details associated with demonstration checkouts. This lab prototype implements the principal shopping flow using browser Local Storage. Real payment processing and secure account management require a server-side system.

**5. Modules**

Home and Navigation Module: Displays the StyleHub home page, navigation menu, promotional banner and popular fashion categories.

Category and Subcategory Module: Groups products under Women, Men, Kids, Footwear and Accessories and displays products associated with each category.

Product Search and Details Module: Searches product names and descriptions, filters by category and displays product information and price.

Shopping Cart Module: Adds products, changes quantities, removes items, clears the cart and calculates the order total.

Checkout and Order Module: Collects a demo customer name, creates an order reference and saves the initial order status.

Order Tracking and History Module: Displays previous demo orders, customer name, order total, date and status.

Administrator Module: Allows product creation, price and description modification, and product deletion.

Login Demonstration Module: Shows a basic front-end login prompt. It is not secure production authentication.

**6. Algorithm**

Start the application and read product, cart and order data from Local Storage. Load the default catalogue if no saved product data exists.

Display the home page with popular fashion categories and their associated products.

Allow the customer to search by product name or description and filter products by category.

When Add to Cart is selected, increase the quantity if the product already exists in the cart; otherwise add it as a new entry.

Allow the customer to increase or decrease quantities, remove individual items or clear the cart. Recalculate the total.

When Checkout is selected, verify that the cart contains products and request the customer's name.

Calculate the order total, generate an order reference, save the order and clear the cart.

Display order history and the current order status.

If an administrator adds, edits or deletes a product, validate the input and update the catalogue.

Save all changes to Local Storage and refresh the displayed page sections. Stop.

**7. Complete HTML, CSS and JavaScript Code**

```
<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>StyleHub - Online Fashion Store</title>

<style>

:root{--dark:#292323;--accent:#8b5148;--bg:#f7f5f2;--muted:#746966}

*{box-sizing:border-box}body{margin:0;font-family:Arial,sans-serif;background:var(-
bg);color:#382b29}

header{background:var(--dark);color:white;padding:16px 5%;display:flex;align-
items:center;justifycontent:space-between;gap:14px;flex-wrap:wrap}

.logo{font-size:27px;font-weight:bold;letter-spacing:2px}nav

a{color:white;textdecoration:none;margin:0 8px}

input,select,button{font:inherit;padding:10px;border:1px solid #d5c9c3;border-radius:7px}

button{background:var(--accent);color:white;border:0;cursor:pointer}button:hover{opacity:.88}

main{max-
width:1200px;margin:auto;padding:24px}.hero{background:#eadbd4;padding:38px;borderradius:16px;
margin-bottom:24px}

.hero h1{font-size:35px;margin:0 0 10px}.section{margin:25px 0}

.grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(210px,1fr));gap:16px}

.card{background:white;padding:18px;border-radius:12px;box-shadow:0 2px 8px #29232312}

.category{border:1px solid #e2d8d2}.muted{color:var(--muted);font-size:14px}

.price{font-size:19px;font-weight:bold;color:var(--
accent)}.actions{display:flex;gap:8px;flexwrap:wrap;margin-top:10px}

.admin{background:#fff0e9;padding:18px;border-
radius:12px}footer{textalign:center;padding:24px;color:var(--muted)}

@media(max-width:650px){header{align-items:flex-start}nav{line-height:2}.hero h1{font-size:27px}}

</style>


</head>
```

```
<body>

<header>

<div class="logo">STYLEHUB</div>

<nav><a href="#home">Home</a><a href="#categories">Categories</a><a
href="#products">Products</a><a href="#orders">Orders</a><a href="#admin">Admin</a></nav>

<div><input id="search" placeholder="Search fashion..." oninput="renderProducts()"><button
onclick="login()">Login</button></div>

</header>

<main id="home">

<section class="hero"><h1>Find Your Everyday Style</h1><p>New-season fashion for every moment
and every you.</p><button onclick="document.getElementById('products').scrollIntoView()">Shop
Now</button></section>

<section class="section" id="categories"><h2>Shop by Category</h2><div id="categoryList"
class="grid"></div></section>

<section class="section" id="products"><h2>Trending Now</h2>

<label>Category: <select id="filter"
onchange="renderProducts()"><option>All</option></select></label>

<div id="productList" class="grid" style="margin-top:15px"></div>

</section>

<section class="section card"><h2>Shopping Cart</h2><ul id="cartItems"></ul><p><b>Total: ₹<span
id="total">0</span></b></p>

<div class="actions"><button onclick="checkout()">Checkout</button><button
onclick="clearCart()">Remove All</button></div>

</section>

<section class="section card" id="orders"><h2>Order Tracking & History</h2><div id="orderList">No
orders placed yet.</div></section>

<section class="section admin" id="admin"><h2>Administrator - Manage Products</h2>

<p class="muted">Demonstration admin panel. A real store must protect this area with server-side
authentication and authorization.</p>

<div class="actions"><input id="name" placeholder="Product name"><input id="price" type="number"
min="1" placeholder="Price">
```

<select
id="cat"><option>Women</option><option>Men</option><option>Kids</option><option>Footwear<
/option><option>Accessories</option></select>

<input id="desc" placeholder="Description"><button onclick="addProduct()">Add
Product</button>

</section>

</main>

<footer>© 2026 StyleHub Online Fashion Store</footer>

<script> const

categories=["Women","Men","Kids","Footwear","Accessories"]; const

defaults=[

{id:1,name:"Floral Summer Dress",price:1499,category:"Women",description:"Lightweight floral dress for casual outings."},

{id:2,name:"Classic Linen Shirt",price:1199,category:"Men",description:"Comfortable regular-fit linen shirt."},

{id:3,name:"Kids Cotton Set",price:899,category:"Kids",description:"Soft cotton two-piece outfit for children."},

{id:4,name:"Everyday Sneakers",price:1799,category:"Footwear",description:"Lightweight everyday walking sneakers."},

{id:5,name:"Mini Crossbody Bag",price:999,category:"Accessories",description:"Compact crossbody bag for daily use."}

];

let products=JSON.parse(localStorage.getItem("sh_products"))||defaults;

let cart=JSON.parse(localStorage.getItem("sh_cart"))||[]; let

orders=JSON.parse(localStorage.getItem("sh_orders"))||[];

function
save(){localStorage.setItem("sh_products",JSON.stringify(products));localStorage.setItem("sh_cart",JS
ON.stringify(cart));localStorage.setItem("sh_orders",JSON.stringify(orders))} function init(){

document.getElementById("categoryList").innerHTML=categories.map(c=>`<div class="card
category"><h3>${c}</h3><p

```
class="muted">${products.filter(p=>p.category===c).map(p=>p.name).join(" • ")||"Explore collection"}</p></div>`).join("");

document.getElementById("filter").innerHTML='<option>All</option>'+categories.map(c=>`<option>${c}</option>`).join(""); renderProducts();renderCart();renderOrders();renderAdmin();

} function

renderProducts(){

let term=document.getElementById("search").value.toLowerCase(),cat=document.getElementById("filter").value;

let found=products.filter(p=>(cat==="All"||p.category===cat)&&(p.name.toLowerCase().includes(term)||p.description.toLowerCase().includes(term)));

document.getElementById("productList").innerHTML=found.map(p=>`<article class="card"><h3>${p.name}</h3><p class="muted">${p.category}</p><p>${p.description}</p><p class="price">₹${p.price.toLocaleString("en-IN")}</p><div class="actions"><button onclick="addToCart(${p.id})">Add to Cart</button><button onclick="alert('${p.name.replace(/'/g,"")}\\n${p.description.replace(/'/g,"")}')">Details</button></div></article>`).join("")||"<p>No matching products found.</p>";

}

function addToCart(id){let x=cart.find(i=>i.id===id);if(x)x.qty++;else

cart.push({id,qty:1});save();renderCart();alert("Product added to cart.")} function

renderCart(){

document.getElementById("cartItems").innerHTML=cart.map(x=>{let p=products.find(y=>y.id===x.id);return p?`<li>${p.name} — ₹${p.price} × ${x.qty} <button onclick="qty(${x.id},-1)">−</button> <button onclick="qty(${x.id},1)">+</button> <button onclick="removeItem(${x.id})">Remove</button></li>`:""}).join("")||"<li>Your cart is empty.</li>";

document.getElementById("total").textContent=cart.reduce((s,x)=>s+(products.find(p=>p.id===x.id)?.price||0)*x.qty,0).toLocaleString("en-IN");

}

function qty(id,n){let x=cart.find(i=>i.id===id);if(x)x.qty+=n;cart=cart.filter(i=>i.qty>0);save();renderCart()} function removeItem(id){cart=cart.filter(x=>x.id!==id);save();renderCart()} function clearCart(){cart=[];save();renderCart()} function checkout(){ if(!cart.length){alert("Your cart is empty.");return} let customer=prompt("Enter customer name for demo checkout:");if(!customer)return; let total=cart.reduce((s,x)=>s+(products.find(p=>p.id===x.id)?.price||0)*x.qty,0);
```

```
orders.unshift({number:"SH"+Date.now().toString().slice(-7),customer,total,status:"Order Placed",date:new Date().toLocaleString()}); cart=[];save();renderCart();renderOrders();alert("Demo order placed successfully.");
}
function renderOrders(){document.getElementById("orderList").innerHTML=orders.map(o=>`<p><b>${o.number}</b> — ${o.customer} — ₹${o.total.toLocaleString("en-IN")} — ${o.status}<br><small>${o.date}</small></p>`).join("")||"No orders placed yet."} function addProduct(){
let name=document.getElementById("name").value.trim(),price=Number(document.getElementById("price").value),category=document.getElementById("cat").value,description=document.getElementById("desc").value.trim(); if(!name||price<=0||!description){alert("Enter a product name, valid price and description.");return} products.push({id:Date.now(),name,price,category,description});save();init();
document.getElementById("name").value="";document.getElementById("price").value="";document.getElementById("desc").value="";
}
function renderAdmin(){document.getElementById("adminList").innerHTML="<h3>Current products</h3>"+products.map(p=>`<p>${p.name} — ₹${p.price} <button onclick="editProduct(${p.id})">Edit</button> <button onclick="deleteProduct(${p.id})">Delete</button></p>`).join("")}
function editProduct(id){let p=products.find(x=>x.id===id),price=prompt("Enter updated price:",p.price),desc=prompt("Enter updated description:",p.description);if(price!==null&&Number(price)>0&&desc!==null){p.price=Number(price);p.description=desc;save();init()}}
function deleteProduct(id){if(confirm("Delete this product?")){products=products.filter(p=>p.id!==id);cart=cart.filter(x=>x.id!==id);save();init()}}
function login(){let user=prompt("Enter username (demo login):");if(user)alert("Welcome, "+user+". Demo login only.")}
init();
</script>
</body>
</html>
```

## 8. Sample Input

The following sample values can be used to test the application:

Search term: Floral Summer Dress

Category filter: Women

Selected product: Floral Summer Dress

Quantity: 2

Customer name at checkout: Priya

Administrator adds: Denim Jacket | Price: 2199 | Category: Men | Description: Casual blue denim jacket.

## 9. Expected Output with Image

The expected home page contains the StyleHub header and navigation links, a search field, promotional banner, category cards and trending products with prices and Add to Cart buttons. The cart displays selected products, editable quantities and the calculated total. A successful demonstration checkout creates an order record that is shown in the order history.

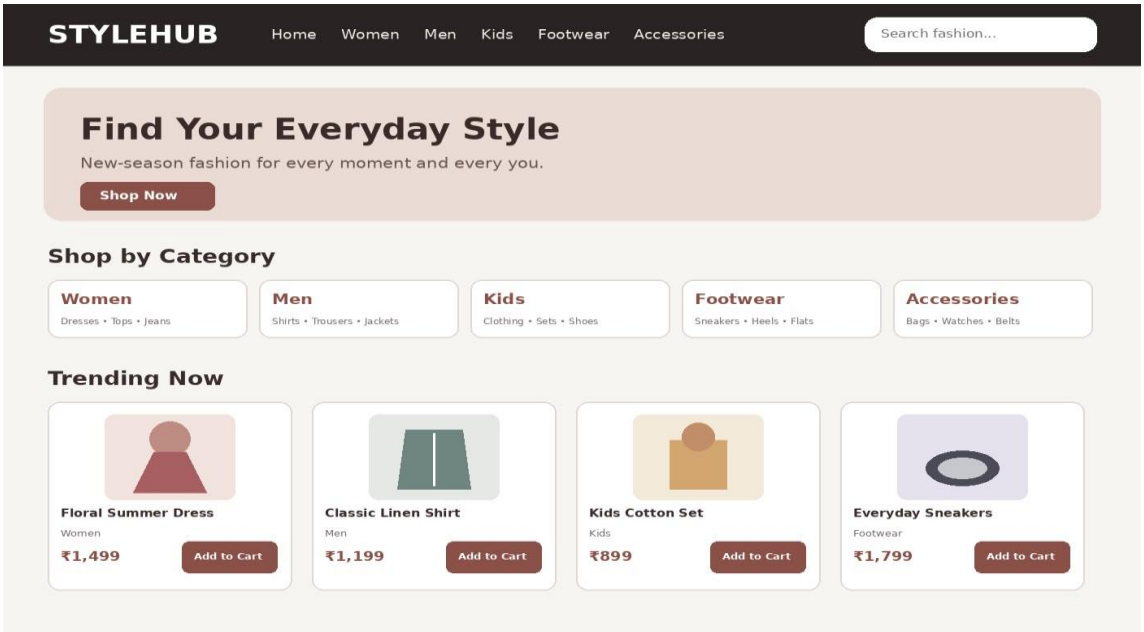



*Figure 1: Expected home page output of the StyleHub online fashion shopping platform.*

10. Result

Thus, the StyleHub online fashion shopping platform was designed and implemented successfully using HTML, CSS and JavaScript. The prototype demonstrates category-based navigation, product search and filtering, product details, shopping cart operations, checkout, order history and administrator product management. Browser Local Storage preserves the demonstration catalogue, cart and order information between page refreshes.

# Experiment-11

# Home Finder  Online Real Estate Platform

*HTML, CSS and JavaScript*

**1. Title**

Design and Development of Home Finder – An Online Real Estate Property Search Platform using HTML, CSS and JavaScript.

**2. Aim**

To develop a user-friendly real estate website that helps buyers search for houses, apartments, land and commercial properties according to location, property type, budget and number of bedrooms. The system also demonstrates saved search criteria, property details, map navigation, agent search, customer visit records and administrator property management.

**3. Software Requirements**

Operating System: Windows 10/11, Linux or macOS

Code Editor: Visual Studio Code / Notepad++

Languages: HTML5, CSS3 and JavaScript (ES6)

Browser: Google Chrome, Microsoft Edge or Firefox

Storage: Browser Local Storage for this lab prototype

Internet connection: Required to open Google Maps links

Hardware: Computer with 4 GB RAM or above

**4. Problem Description**

Buyers and renters often need to search multiple websites or contact several agents to find properties that meet their requirements. Home Finder provides a single interface for browsing property listings and filtering them by area, price, type and bedrooms. Buyers can save search criteria and reuse them instead of entering the same information repeatedly. The prototype also provides property details, Google Maps search links, agent/company search, and a basic visit log. Administrators can add, edit and remove property records. The supplied lab implementation uses Local Storage for demonstration data; a production system requires a secure database and backend.

**5. Modules**

**Buyer Login Demonstration:** Provides a basic buyer login prompt. It demonstrates the entry point but does not implement secure account creation or password storage.

**Property Search and Filter Module:** Filters property listings using location, property type, maximum budget and minimum bedroom count.

**Saved Search Module:** Stores the buyer's search criteria in Local Storage and allows the criteria to be reused or deleted.

**Property Details Module:** Displays property title, location, type, price and description.

**Google Maps Module:** Opens a Google Maps search for the selected property's area.

**Agent and Company Search Module:** Searches the sample agent directory by agent name, company or service area.

**Customer Visit Record Module:** Records page visits with date and time in the current browser for demonstration purposes.

**Administrator Module:** Allows the administrator to add properties, update prices and descriptions, and remove listings.

**6. Algorithm**

Start the Home Finder application and load property listings, saved searches and visit records from Local Storage.

Record the current page visit with its date and time.

Display the home page and available property listings.

Accept buyer search criteria such as location, property type, maximum budget and number of bedrooms.

Filter the property list and display matching records.

Allow the buyer to save the current search criteria or reuse a previously saved search.

When a property is selected, display its details and provide a Google Maps link for its area.

Allow the buyer to search the sample agent/company directory.

If the administrator adds, edits or removes a property, validate the input and update the property catalogue.

Save updated records in Local Storage, refresh the displayed information and stop.

**7. Complete HTML, CSS and JavaScript Code**

<!DOCTYPE html>

```
<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Home Finder - Real Estate Portal</title>

<style>

:root{--navy:#17324d;--green:#287a68;--bg:#f3f6f8;--muted:#607784}

*{box-sizing:border-box}body{margin:0;font-family:Arial,sans-serif;background:var(-
bg);color:#263d4a}

header{background:var(--navy);color:white;padding:16px 5%;display:flex;justify-
content:spacebetween;align-items:center;gap:14px;flex-wrap:wrap}

.logo{font-size:26px;font-weight:bold}nav a{color:white;text-decoration:none;margin:0 8px}

button,input,select{font:inherit;padding:10px;border:1px solid #c7d3da;border-radius:7px}

button{background:var(--green);color:white;border:0;cursor:pointer}button:hover{opacity:.9}

main{max-
width:1200px;margin:auto;padding:24px}.hero{background:#dce9e6;padding:35px;borderradius:16px}

.hero h1{font-size:34px;margin:0 0 10px;color:var(--navy)}

.filters,.card{background:white;padding:18px;border-radius:12px;box-shadow:0 2px 8px #17324d12}

.filters{display:grid;grid-template-columns:repeat(auto-fit,minmax(170px,1fr));gap:12px;margin:22px
0}

.grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(260px,1fr));gap:18px}

.card h3{margin:8px 0}.muted{color:var(--muted);font-size:14px}.price{font-
size:20px;fontweight:bold;color:var(--green)}

.section{margin:25px 0}.actions{display:flex;gap:8px;flex-wrap:wrap;margin-top:10px}

.admin{background:#eef5f1;padding:18px;border-
radius:12px}footer{textalign:center;padding:25px;color:var(--muted)} @media(max-
width:650px){header{align-items:flex-start}.hero h1{font-size:27px}}

</style>

</head>
```

```
<body>

<header>

<div class="logo">HOME FINDER</div>

<nav><a href="#home">Home</a><a href="#properties">Properties</a><a href="#saved">Saved Searches</a><a href="#agents">Agents</a><a href="#admin">Admin</a></nav>

<button onclick="login()">Login / Register</button>

</header>

<main id="home">

<section class="hero"><h1>Find a Place to Call Home</h1><p>Explore homes and land that match your location and budget.</p></section>

<section class="filters">

<label>Location<input id="location" placeholder="Enter area"></label>

<label>Property Type<select id="type"><option>All</option><option>House</option><option>Apartment</option><option>Land</option><option>Commercial</option></select></label>

<label>Maximum Budget (₹)<input id="budget" type="number" placeholder="e.g. 5000000"></label>

<label>Bedrooms<select id="beds"><option value="0">Any</option><option value="1">1+</option><option value="2">2+</option><option value="3">3+</option></select></label>

<button onclick="searchProperties()">Search</button><button onclick="saveSearch()">Save Search</button>

</section>

<section class="section" id="properties"><h2>Featured Properties</h2><div id="propertyList" class="grid"></div></section>

<section class="section card" id="saved"><h2>Saved Searches</h2><div id="savedList">No saved searches.</div></section>

<section class="section card" id="agents"><h2>Agent and Company Search</h2><input id="agentSearch" placeholder="Search agent or company" oninput="renderAgents()"><div id="agentList"></div></section>

<section class="section card"><h2>Customer Visit Records</h2><p id="visitCount">Visits recorded in this browser: 0</p><div id="visitList"></div></section>
```

<section class="section admin" id="admin"><h2>Administrator - Manage Properties</h2>

Demo admin panel. Protect real administration features with server-side authentication.

<input id="pname" placeholder="Property title"><input id="pprice" type="number" placeholder="Price">

<input id="parea" placeholder="Area / Location"><select id="ptype"><option>House</option><option>Apartment</option><option>Land</option><option>Commercial</option></select>

<input id="pbeds" type="number" min="0" placeholder="Bedrooms"><input id="pdesc"

placeholder="Description"><button onclick="addProperty()">Add Property</button> <div

id="adminList">

</section>

</main>

<footer>© 2026 Home Finder Real Estate Portal</footer>

<script> const

initial=[

{id:1,title:"Family House",price:4200000,area:"Devanakonda",type:"House",beds:2,desc:"Comfortable family home with two bedrooms."},

{id:2,title:"Residential Plot",price:1800000,area:"Kurnool",type:"Land",beds:0,desc:"Residential plot suitable for a new home."},

{id:3,title:"Modern Villa",price:7800000,area:"Nandyal",type:"House",beds:3,desc:"Spacious villa with modern facilities."},

{id:4,title:"City Apartment",price:3500000,area:"Kurnool",type:"Apartment",beds:2,desc:"Twobedroom apartment close to local services."}

];

let properties=JSON.parse(localStorage.getItem("hf_properties"))||initial; let

saved=JSON.parse(localStorage.getItem("hf_saved"))||[];

let visits=JSON.parse(localStorage.getItem("hf_visits"))||[];

const agents=[{name:"Ravi Kumar",company:"HomeFirst Realty",area:"Kurnool"},{name:"Anitha Rao",company:"Prime Estates",area:"Nandyal"},{name:"City Homes Team",company:"City Homes",area:"Devanakonda"}];

```
function
persist(){localStorage.setItem("hf_properties",JSON.stringify(properties));localStorage.setItem("hf_sav
ed",JSON.stringify(saved));localStorage.setItem("hf_visits",JSON.stringify(visits))} function
recordVisit(){visits.unshift({time:new Date().toLocaleString(),page:"Home
page"});visits=visits.slice(0,20);persist();document.getElementById("visitCount").textContent="Visits
recorded in this browser:
"+visits.length;document.getElementById("visitList").innerHTML=visits.slice(0,5).map(v=>`<p>${v.pa
ge} — ${v.time}</p>`).join("")} function renderProperties(list=properties){

document.getElementById("propertyList").innerHTML=list.map(p=>`<article
class="card"><h3>${p.title}</h3><p class="muted">${p.area} • ${p.type} • ${p.beds}
bedrooms</p><p>${p.desc}</p><p class="price">₹${p.price.toLocaleString("en-IN")}</p><div
class="actions"><button onclick="details(${p.id})">View Details</button><button
onclick="favorite(${p.id})">Save</button><button onclick="mapArea('${p.area}')">View
Map</button></div></article>`).join("")||"<p>No matching properties found.</p>";

} function

searchProperties(){

let
area=document.getElementById("location").value.toLowerCase(),type=document.getElementById("typ
e").value,budget=Number(document.getElementById("budget").value)||Infinity,beds=Number(documen
t.getElementById("beds").value);

renderProperties(properties.filter(p=>(!area||p.area.toLowerCase().includes(area))&&(type==="All"||p.t
ype===type)&&p.price<=budget&&p.beds>=beds));

} function

saveSearch(){

let
criteria={area:document.getElementById("location").value,type:document.getElementById("type").valu
e,budget:document.getElementById("budget").value,beds:document.getElementById("beds").value};

saved.unshift(criteria);persist();renderSaved();alert("Search criteria saved."); }


function
renderSaved(){document.getElementById("savedList").innerHTML=saved.map((s,i)=>`<p>${s.area||"A
ny area"} | ${s.type} | Budget: ${s.budget||"Any"} | Beds: ${s.beds} <button
onclick="reuseSearch(${i})">Use Search</button> <button
onclick="deleteSearch(${i})">Delete</button></p>`).join("")||"No saved searches."}

function reuseSearch(i){let
s=saved[i];document.getElementById("location").value=s.area;document.getElementById("type").value
```

=s.type;document.getElementById("budget").value=s.budget;document.getElementById("beds").value=s.beds;searchProperties()} function deleteSearch(i){saved.splice(i,1);persist();renderSaved()}

function favorite(id){let p=properties.find(x=>x.id===id);alert("Saved to favorites: "+p.title+" (demo).")}

function details(id){let p=properties.find(x=>x.id===id);alert(p.title+"\\nLocation: "+p.area+"\\nType: "+p.type+"\\nPrice: ₹"+p.price+"\\n"+p.desc)}

function mapArea(area){let url="https://www.google.com/maps/search/?api=1&query="+encodeURIComponent(area);window.open(url,"_blank")}

function renderAgents(){let q=document.getElementById("agentSearch").value.toLowerCase();document.getElementById("agentList").innerHTML=agents.filter(a=>(a.name+" "+a.company+" "+a.area).toLowerCase().includes(q)).map(a=>`<p><b>${a.name}</b> — ${a.company} — ${a.area}</p>`).join("")||"No matching agents."}

function login(){let user=prompt("Enter buyer username (demo login):");if(!user)return;let pass=prompt("Enter password (demo only):");if(pass)alert("Welcome, "+user+". This prototype does not store passwords or provide secure authentication.")} function addProperty(){

let title=document.getElementById("pname").value.trim(),price=Number(document.getElementById("pprice").value),area=document.getElementById("parea").value.trim(),type=document.getElementById("ptype").value,beds=Number(document.getElementById("pbeds").value)||0,desc=document.getElementById("pdesc").value.trim(); if(!title||price<=0||!area||!desc){alert("Enter title, valid price, location

and description.");return}

properties.push({id:Date.now(),title,price,area,type,beds,desc});persist();renderProperties();renderAdmin();alert("Property added.");

}

function editProperty(id){let p=properties.find(x=>x.id===id),price=prompt("New price:",p.price),desc=prompt("New description:",p.desc);if(price!==null&&Number(price)>0&&desc!==null){p.price=Number(price);p.desc=desc;persist();renderProperties();renderAdmin()}}

function removeProperty(id){if(confirm("Remove this property?")){properties=properties.filter(p=>p.id!==id);persist();renderProperties();renderAdmin()}}

function renderAdmin(){document.getElementById("adminList").innerHTML="<h3>Property records</h3>"+properties.map(p=>`<p>${p.title} — ₹${p.price} <button onclick="editProperty(${p.id})">Edit</button> <button

onclick="removeProperty(${p.id})">Delete</button></p>`).join("")}
recordVisit();renderProperties();renderSaved();renderAgents();renderAdmin(); </script>

</body>

</html>

**8. Sample Input**

Use the following sample values to test the website:

Location: Devanakonda

Property Type: House

Maximum Budget: 5000000

Bedrooms: 2 or more

Saved search: Devanakonda | House | ₹50,00,000 | 2+ bedrooms

Agent search keyword: HomeFirst

Administrator adds: Green Valley Plot | ₹2,500,000 | Kurnool | Land | 0 bedrooms | Residential plot near main road.

**9. Expected Output with Image**

The home page displays the Home Finder title, navigation menu, property search form, featured property cards, prices and action buttons. Search results are filtered according to the entered criteria. Buyers can save and reuse searches, open the property area in Google Maps, search sample agents and view the browser visit log. The administrator section displays property records and management controls.

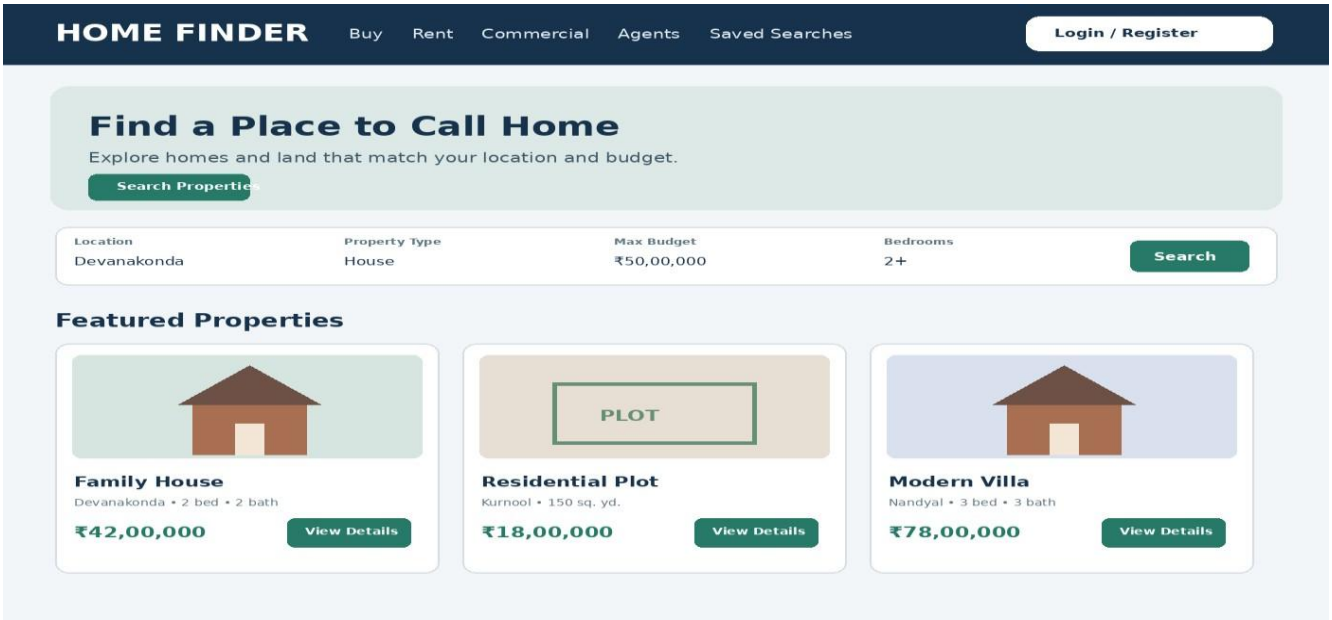



*Figure 1: Expected home page output of the Home Finder real estate platform.*

**10. Result**

Thus, the Home Finder real estate property search website was designed and implemented successfully using HTML, CSS and JavaScript. The prototype demonstrates property searching and filtering, saved search criteria, property details, Google Maps navigation, agent/company search, customer visit records and administrator property management.

# Experiment-12

# INSUREEASY
# ONLINE INSURANCE MANAGEMENT SYSTEM

**2. Aim**

To design and develop a user-friendly web-based insurance management system that helps customers compare insurance policies, register and manage policy information, view premium details, and receive renewal or payment reminders. The system also provides administrative control over agent registration and supports agents in managing customer and policy records. **3. Software Requirements**

| Requirement | Specification |
|---|---|
| Operating system | Windows 10/11, Linux, or macOS |
| Front-end | HTML5, CSS3, JavaScript |

| | |
|---|---|
| Browser | Google Chrome, Mozilla Firefox, Microsoft Edge |
| Editor | Visual Studio Code / Notepad++ |
| Storage in prototype | Browser localStorage |
| Optional production backend | Node.js / PHP / Python with MySQL or another database |
| Optional SMS service | SMS gateway/API such as a licensed provider |

**4. Problem Description**

In a manual insurance process, customer, agent, policy, premium, and commission records may be maintained in paper files or separate spreadsheets. Searching, updating, calculating dues, tracking renewals, and notifying policyholders can therefore take considerable time and may lead to missing or outdated information. Customers may also find it difficult to compare policy features in one place.

InsureEasy proposes a centralized online platform. Administrators can approve agent accounts and view agent information. Agents can register, create customer records, maintain customer and policy details, and initiate due-date reminders. Customers can sign in to view and manage their policy information and receive alerts. The included HTML/CSS/JavaScript implementation is a front-end demonstration; real authentication, payments, SMS delivery, and secure multi-user data storage require a server, database, and external service integrations.

**5. Modules**

**5.1 Admin Module**

Admin login with role-based access (planned for the full system).

Approve or reject agent login/registration requests.

View agent details and monitor agent activity.

**5.2 Agent Module**

Agent registration and login.

Create customer records and manage customer details.

Add, view, and update policy details.

Review premium due dates and prepare SMS alerts for customers.

5.3 User / Customer Module

Customer login (planned for the full system).

View and manage existing policy details.

View premium and renewal information.

Receive SMS alerts for upcoming or overdue payments.

**6. Algorithm**

Start the application and display the InsureEasy home page.

Display available insurance categories: health, auto, and home.

When a visitor selects a policy, place the selected category in the registration form.

Validate the customer name and email address.

If the input is invalid, display an error message and request corrected details.

If the input is valid, create a customer record and save the demo record in browser localStorage.

Display the customer record in the dashboard table.

When the reminder button is selected, display a demo reminder confirmation.

In a production deployment, send the reminder through a server-side SMS gateway after checking the policy due date and customer consent.

Stop.

**7. Complete HTML, CSS and JavaScript Code**

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>InsureEasy | Insurance Management</title>

<style>

* { box-sizing: border-box; } body { margin: 0; font-family: Arial, sans-serif;

background: #f4f7fb; color: #1f2937; } header { background: #123b68; color: white;

padding: 18px 6%; display:flex; justify-content:space-between; align-items:center;

```
gap:16px; flex-wrap:wrap; } header h1 { margin:0; font-size:24px; } header nav a {
color:white; text-decoration:none; margin-left:18px; font-size:14px; }
.container { width:min(1100px, 92%); margin:28px auto; }
.hero { background:linear-gradient(120deg,#155799,#1595a3); color:white; padding:30px; border-
radius:16px; }
.hero h2 { margin:0 0 8px; font-size:28px; }
.hero p { max-width:650px; line-height:1.55; }
.actions { display:flex; gap:10px; flex-wrap:wrap; margin-top:18px; } button,
.btn { border:0; border-radius:8px; padding:11px 16px; cursor:pointer; font-
weight:bold; background:#ffcf5a; color:#17324d; }
.btn.secondary { background:white; color:#155799; }
.grid { display:grid; grid-template-columns:repeat(3,1fr); gap:16px; margin-top:22px; }
.card { background:white; border-radius:13px; padding:20px; box-shadow:0 4px 16px #142c4a12; }
.card h3 { margin:0 0 8px; color:#174a78; }
.muted { color:#667085; font-size:14px; line-height:1.5; }
.price { font-size:23px; font-weight:bold; margin:12px 0; color:#123b68; } .panel {
background:white; border-radius:13px; padding:20px; margin-top:22px; box-shadow:0
4px 16px #142c4a12; } label { display:block; font-size:13px; font-weight:bold;
margin:12px 0 5px; } input, select { width:100%; padding:11px; border:1px solid
#d0d7e2; border-radius:8px; }
.form-row { display:grid; grid-template-columns:1fr 1fr; gap:14px; } table {
width:100%; border-collapse:collapse; margin-top:12px; font-size:14px; } th,td {
text-align:left; padding:11px; border-bottom:1px solid #e8edf3; } th {
color:#174a78; background:#f7f9fc; }
.status { color:#16804a; font-weight:bold; }
#message { margin-top:12px; color:#137547; font-weight:bold; } footer {
text-align:center; color:#667085; padding:26px; font-size:13px; }
```

```
@media(max-width:760px) {
.grid { grid-template-columns:1fr; } .form-row
{ grid-template-columns:1fr; } header nav a {
margin:0 12px 0 0; }
.hero h2 { font-size:23px; }
}
</style>
</head>
<body>
<header>
<h1>InsureEasy</h1>
<nav><a href="#policies">Policies</a><a href="#dashboard">Dashboard</a><a
href="#register">Register</a></nav>
</header>
<main class="container">
<section class="hero">
<h2>Insurance made simple.</h2>
<p>Compare plans, manage your policies and keep track of premium due dates from one convenient
dashboard.</p>
<div class="actions">
<a class="btn" href="#policies">Compare policies</a>
<a class="btn secondary" href="#register">Create account</a>
</div>
</section>
<section id="policies">
<h2>Explore insurance plans</h2>
<div class="grid">
```

<article class="card">

<h3>Health Insurance</h3> Hospitalisation and medical expense coverage.

From ₹650/month <button onclick="selectPolicy('Health Insurance')">Select plan</button>

</article>

<article class="card">

<h3>Auto Insurance</h3> Protection for your vehicle against covered risks.

From ₹480/month <button onclick="selectPolicy('Auto Insurance')">Select plan</button>

</article>

<article class="card">

<h3>Home Insurance</h3> Cover for your home and eligible belongings.

From ₹390/month <button onclick="selectPolicy('Home Insurance')">Select plan</button>

</article>

</section>

<section id="register" class="panel">

<h2>Customer registration</h2>

<label for="customerName">Full name</label><input id="customerName" placeholder="Enter customer name">

<label for="customerEmail">Email address</label><input id="customerEmail" type="email" placeholder="name@example.com">

<label for="policyType">Selected policy</label>

<select id="policyType">

<option>Health Insurance</option><option>Auto Insurance</option><option>Home Insurance</option>

```
</select>

<div class="actions"><button onclick="addCustomer()">Create customer</button></div>

<p id="message" aria-live="polite"></p>

</section>

<section id="dashboard" class="panel">

<h2>Customer dashboard</h2>

<p class="muted">Demo view: records are stored in this browser using localStorage.</p> <table>

<thead><tr><th>Customer</th><th>Policy</th><th>Premium due</th><th>Status</th></tr></thead>

<tbody id="customerRows">

<tr><td>Sample Customer</td><td>Health Insurance</td><td>15 Dec 2026</td><td
class="status">Active</td></tr>

</tbody>

</table>

<div class="actions"><button onclick="sendReminder()">Send demo SMS reminder</button></div> <p
id="reminderMessage" class="muted" aria-live="polite"></p>

</section>

</main>

<footer>© 2026 InsureEasy · Student project demonstration</footer>

<script> const rows = document.getElementById("customerRows");

function selectPolicy(policy) {

document.getElementById("policyType").value = policy;

document.getElementById("register").scrollIntoView({behavior:"smooth"});

} function loadCustomers() { const saved =

JSON.parse(localStorage.getItem("insureEasyCustomers") || "[]");

saved.forEach(customer => appendCustomer(customer, false));

} function appendCustomer(customer, save=true)

{ const tr = document.createElement("tr");
```

```
[customer.name, customer.policy, customer.due, "Active"].forEach((value, index) => {
const td = document.createElement("td"); td.textContent = value; if (index === 3)
td.className = "status"; tr.appendChild(td);
}); rows.appendChild(tr); if (save) { const saved =
JSON.parse(localStorage.getItem("insureEasyCustomers") || "[]");
saved.push(customer); localStorage.setItem("insureEasyCustomers",
JSON.stringify(saved));

} } function addCustomer() { const name =
document.getElementById("customerName").value.trim(); const email
= document.getElementById("customerEmail").value.trim(); const
policy = document.getElementById("policyType").value; const message
= document.getElementById("message"); if (!name || !email ||
!email.includes("@")) { message.textContent = "Please enter a name
and a valid email address."; return; } const customer = {name, email,
policy, due:"15 Dec 2026"}; appendCustomer(customer);
message.textContent = "Customer record added successfully (demo).";
document.getElementById("customerName").value = "";
document.getElementById("customerEmail").value = "";
} function sendReminder() {
document.getElementById("reminderMessage").textContent =
"Demo reminder prepared. Connect an SMS gateway and backend to send a real SMS.";
} loadCustomers();
</script>
</body>
</html>
```

**8. Sample Input**

| Field | Sample value |
|---|---|
| Full name | Ananya Rao |
| Email address | ananya@example.com |
| Selected policy | Health Insurance |
| Premium due date (demo) | 15 Dec 2026 |

Action: Select “Create customer”. Then select “Send demo SMS reminder” in the dashboard.

**9. Expected Output with Image**

The home page displays the InsureEasy navigation bar, a welcome banner, three insurance plan cards, and a customer dashboard. After valid sample details are submitted, a new customer row appears in the dashboard. The reminder button displays a message indicating that a demo reminder has been prepared.

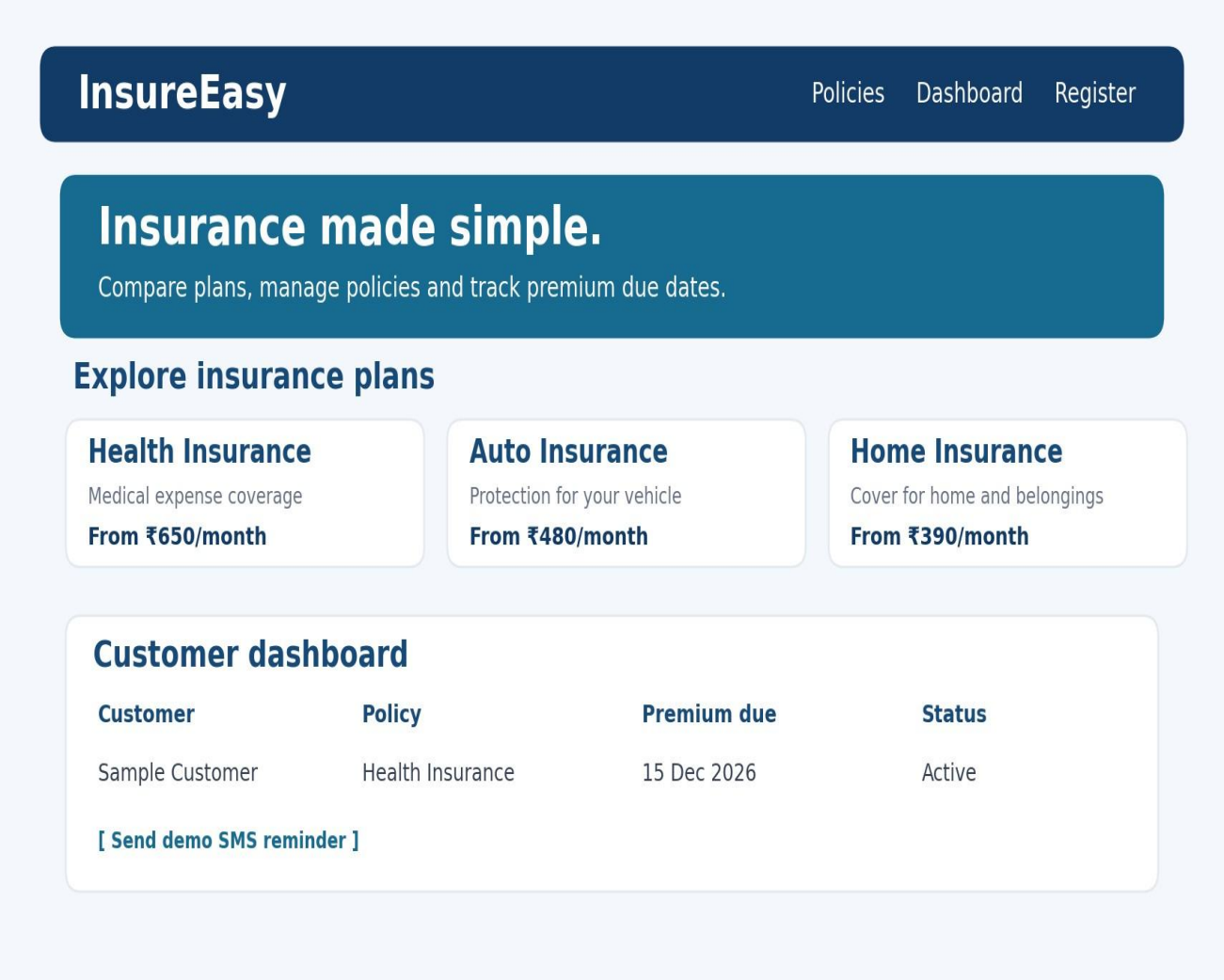



Figure 1: Illustrative InsureEasy home page and customer dashboard.

**10. Result**

The InsureEasy front-end prototype was designed using HTML, CSS, and JavaScript. It demonstrates policy category selection, customer registration with basic validation, dashboard record display, browser-based demo storage, and a simulated SMS reminder. The project provides a foundation for a complete insurance management system. For real-world use, it must be extended with secure server-side authentication, database storage, agent approval workflows, payment processing, policy/claim management, and an SMS gateway integration.